Python Notes: Let's Code a Simple Python Calculator! 🧮

Video: Let's code a simple python calculator! 🧮

Video URL: https://www.youtube.com/watch?v=yUrYouDQZL8

Level: Beginner

Prerequisites: Variables, `input()`, type casting, arithmetic operators, and `if-elif-else` statements.

---

1. Introduction

A **calculator program** is a beginner-friendly Python project that performs arithmetic operations based on the operator selected by the user.

In this project, the user enters:

- An arithmetic operator.
- The first number.
- The second number.

Python then checks the selected operator, performs the appropriate calculation, and displays the result.

2. Step 1: Accept the Operator

First, ask the user which arithmetic operation they want to perform.

```
operator = input("Enter an operator (+ - * /): ")
```

Explanation

- `input()` displays a message and waits for the user's response.
- The selected operator is stored in the variable `operator`.
- Since `input()` returns a string, the operator is stored as text.

For example:

```
Enter an operator (+ - * /): +
```

The variable now contains:

```
operator = "+"
```

Python will later compare this value with the supported operators.

> **Most Important**
>
> The user must enter the correct symbol. The word `"plus"` or the word `"addition"` is not the same as the symbol `"+"`.

---

3. Step 2: Accept the Two Numbers

Next, create two variables to store the numbers that will be used in the calculation.

Why did Python print `1011` instead of `21`?

Because both variables contain strings. When the `+` operator is used with strings, it joins them together. This is called **string concatenation**.

Solution: Type casting with `float()`

Use `float()` to convert the input strings into numeric values.

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

print(num1 + num2)
```

Input:

```
Enter the first number: 10
Enter the second number: 11
```

Output:

```
21.0
```

Now Python performs numerical addition.

Why use `float()` instead of `int()`?

Using `float()` allows the calculator to accept both whole numbers and decimal numbers.

For example:

```
num1 = float("5.5")
num2 = float("6.9")

print(num1 + num2)
```

For the calculator, the conditions compare the `operator` variable with the four supported symbols.

Condition	Calculation
`operator == "+"`	`num1 + num2`
`operator == "-"`	`num1 - num2`
`operator == "*"`	`num1 * num2`
`operator == "/"`	`num1 / num2`
No condition matches	*Display an invalid-operator message*

Remember

`==` is the comparison operator. It checks whether two values are equal. A single `=` is used for assignment.

5. Step 4: Perform Each Arithmetic Operation

5.1 Addition

```
if operator == "+":
    result = num1 + num2
```

If the user selects `+`, Python adds the two numbers and stores the answer in `result`.

Example:

5.3 Multiplication

```
elif operator == "*":
    result = num1 * num2
```

Example:

```
num1 = 3.14
num2 = 3.14

result = num1 * num2
print(result)
```

Output:

```
9.8596
```

The asterisk (`*`) is Python's multiplication operator.

5.4 Division

```
elif operator == "/":
    result = num1 / num2
```

Example:

```
num1 = 420.0
num2 = 69.0

result = num1 / num2
print(result)
```

7. Step 6: Round the Result

Division and some other floating-point calculations may produce long decimal representations.

Python provides the built-in `round()` function to round a number.

Syntax

```
round(number, ndigits)
```

- `number` : the value to round.
- `ndigits` : the number of decimal places to retain.

Example 1: Round to the nearest integer

```
result = 6.086956521?

print(round(result))
```

Output:

```
6
```

Example 2: Round to three decimal places

```
result = 6.086956521?

print(round(result, 3))
```

Output:

8. Step 7: Handle Invalid Operators

What happens if the user enters something that is not a supported operator?

For example:

```
Enter an operator (+ - * /): pizza
```

None of the supported conditions matches the string `"pizza"`.

An `else` statement can handle this situation.

```
if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    result = num1 / num2
else:
    print(f"{operator} is not a valid operator.")
```

Output:

```
pizza is not a valid operator.
```

Key concept

`else` acts as the default branch when none of the preceding conditions is true.

Sample execution 1: Addition

```
Enter an operator (+ - * /): +
Enter the first number: 5.5
Enter the second number: 6.9
Result: 12.4
```

Sample execution 2: Subtraction

```
Enter an operator (+ - * /): -
Enter the first number: 420
Enter the second number: 0.69
Result: 419.31
```

Sample execution 3: Multiplication

```
Enter an operator (+ - * /): *
Enter the first number: 3.14
Enter the second number: 3.14
Result: 9.8596
```

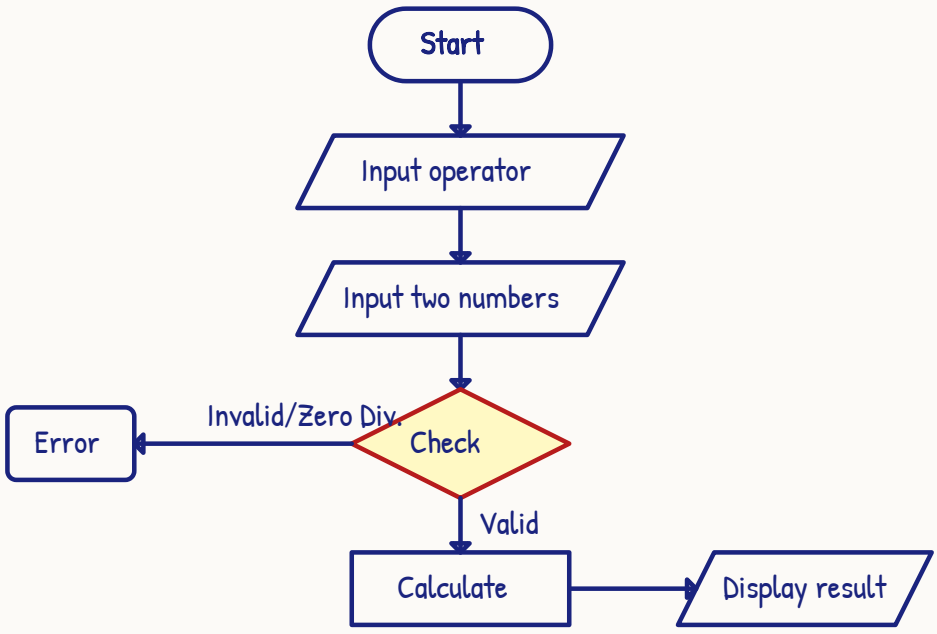
Sample execution 4: Division

```
Enter an operator (+ - * /): /
Enter the first number: 420
Enter the second number: 69
Result: 6.0869565217391304
```

10. Understanding the Program's Flow

The calculator follows a simple decision-making process:

Start → Input operator → Input two numbers → Check operator → Calculate → Display result



If the operator is invalid, the program displays an error message instead of performing a calculation.

If the operator is division, the program also checks whether the second number is zero before dividing.

This is a good example of combining variables, user input, type casting, arithmetic operators, and conditionals in one practical program.

Mistake 2: Using the wrong multiplication symbol

In Python, multiplication uses `*`, not `x`.

Incorrect:

```
result = num1 x num2
```

Correct:

```
result = num1 * num2
```

Mistake 3: Confusing assignment and comparison

Incorrect:

```
if operator = "+":
```

Correct:

```
if operator == "+":
```

Mistake 4: Forgetting a colon

Every `if`, `elif`, and `else` header needs a colon.

Correct:

```
if operator == "+":
    result = num1 + num2
```

Exercise A: Predict the output

Question 1

```
num1 = 10.0
num2 = 5.0
print(num1 + num2)
```

Question 2

```
num1 = 10.0
num2 = 5.0
print(num1 - num2)
```

Question 3

```
num1 = 3.0
num2 = 4.0
print(num1 * num2)
```

Question 4

```
print(round(10 / 3, 2))
```

Question 5

13. Final Revision Sheet

Important concepts

Concept	Remember
`input()`	Accepts user input as a string.
`float()`	Converts valid numeric input into a floating-point number.
`if`	Checks the first condition.
`elif`	Checks another condition if earlier conditions are false.
`else`	Handles cases where no preceding condition matches.
`==`	Checks whether two values are equal.
`*`	Multiplication operator.
`/`	Division operator.
`round()`	Rounds a numeric value to a specified number of digits.
`ZeroDivisionError`	Raised when dividing by zero.

Calculator structure to memorise

```
operator = input("Enter an operator (+ - * /): ")

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    if num2 == 0:
        print("Cannot divide by zero.")
        result = None
    else:
        result = num1 / num2
else:
    print("Invalid operator.")
    result = None

if result is not None:
    print(f"Result: {result}")
```

Final takeaway

This simple calculator is an excellent beginner Python project because it brings together several fundamental concepts:

- Variables to store data.
- `input()` to accept user choices and numbers.
- `float()` to convert numeric input.
- Arithmetic operators to perform calculations.

- `if`, `elif`, and `else` to select the operation.
- `round()` to control numerical precision.
- Error handling to make the program more reliable.

Practice tip

Build the calculator from memory, test all four operations, and then add modulus, floor division, and input validation. The goal is not just to copy the code, but to understand why each statement is needed.